



Introduction to Advanced Java Technology

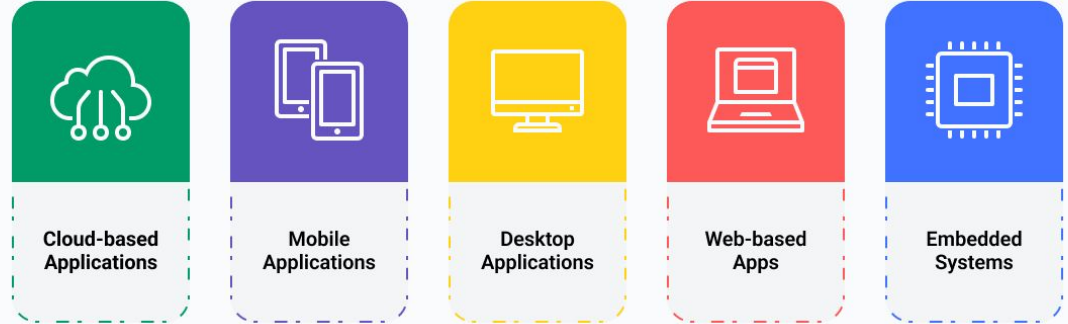
Ms.Shweta Rajput

Department of Computer Science & Engineering

Outline

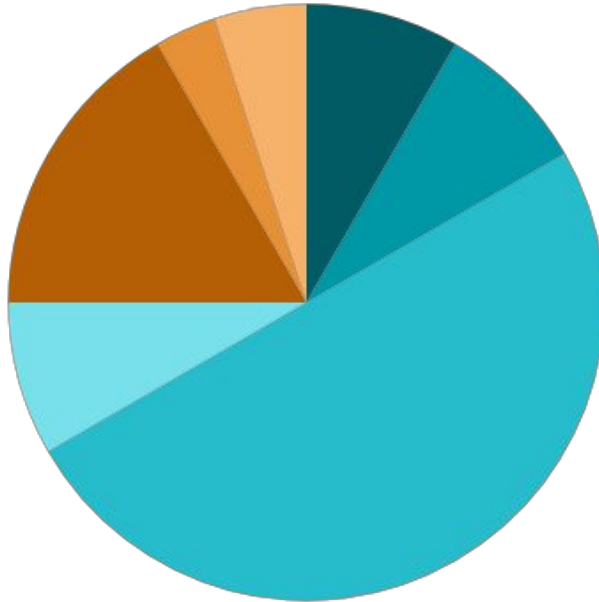
1. Overview
2. Syllabus
3. Course Outcome
4. CEC, Practicals
5. Books
6. Google Classroom

Applications of Java



Source of Image: [/https://softjournal.com/insights/is-java-still-used](https://softjournal.com/insights/is-java-still-used)

Lecture Structure



05 Minutes

Outline & Objectives



05 Minutes

Recap of Previous Lecture



30 Minutes

Delivery of Actual Content as per Lesson Plan



05 Minutes

Industry Relevance



10 Minutes

Interactive Activities



02 Minutes

Importance of Discipline



03 Minutes

Recording of Attendance

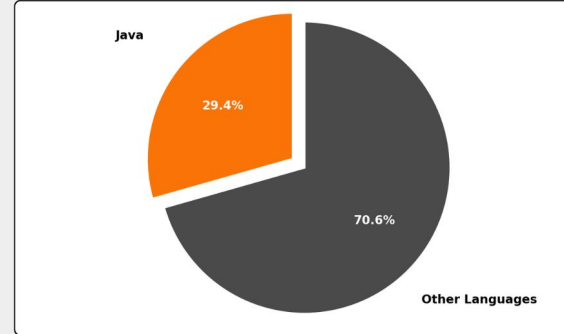
Reference: Office Order No. Provost/092023135, Date: 18th September 2023



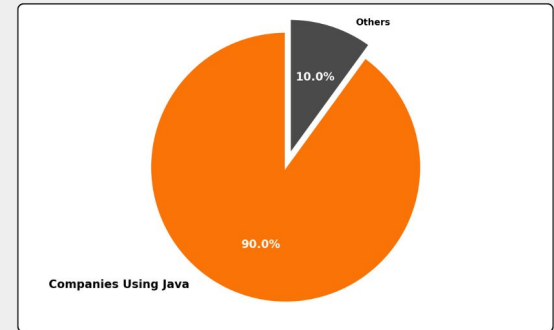
Objectives

1. Introduction of Course
2. Importance
3. Applications in Industry
4. Placement

Secondtalent

Java's Global Market Share (Statista Report)

Secondtalent

Fortune 500 Companies Using Java

Source of Image:

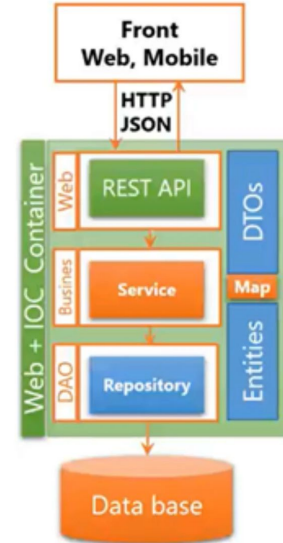
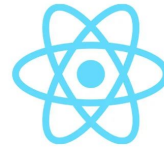
https://fiverr-res.cloudinary.com/images/q_auto,f_auto/gigs2/262240474/original/d603d435f125dddb9b661252a85c0fe17cd388db/make-java-jee-spring-jpa-hibernate-maven-web-mvc-application.png

Introduction of Course

1. J2EE - Java Enterprise Application
2. MVC Architecture
3. Application Server
4. Package
 - java.net
 - Java.sql
 - Javax.servlet
 - Javax.servlet.http
5. Frame Work
 - Hibernate
 - Spring Boot



{ **REST:API** }





Applications in Industry

1. **Mobile Development:** The foundation for native Android app development.
2. **Cloud Computing:** Creating cloud-native apps, microservices, and big data solutions (e.g., for Hadoop).
3. **Big Data & Analytics:** Processing massive datasets for real-time analysis.
4. **Internet of Things (IoT):** Programming sensors, smart devices, and industrial systems (e.g., Bosch, Siemens).
5. **Desktop Applications:** Developing GUI-based software for various operating systems (Windows, Mac, Linux).
6. **Games:** Creating cross-platform and complex 3D games.
7. **Scientific & Financial:** Powering complex simulations and secure trading applications.



Core Development Roles

1. **Java Developer/Software Engineer:** Builds, tests, and maintains Java applications, from enterprise systems to web apps, throughout the development lifecycle.
2. **Backend Developer:** Specializes in server-side logic, databases, and APIs using Java frameworks like Spring.
3. **Android Developer:** Creates applications for the Android platform, often using Java (and Kotlin).
4. **Full-Stack Developer:** Works on both front-end (user interface) and back-end (server-side) aspects of applications.



Specialized & Advanced Roles:

1. **Software Architect:** Designs the high-level structure of complex software systems, making key technical decisions.
2. **DevOps Engineer:** Bridges development and operations, automating deployments and managing CI/CD pipelines for Java apps.
3. **SDET (Software Development Engineer in Test):** Develops automated testing frameworks and code to ensure software quality.
4. **Big Data Developer:** Works with large datasets, often using Java-based tools.
5. **Technical Consultant:** Provides expert advice and designs solutions for clients using deep Java knowledge.



Advanced Java (BTCS603) - 4 Credits

1. Five Units
2. 100 Th (20 Mid+ 40 CEC + 40 External) + 50 Pra
3. CEC - 40M (Quiz - 10 + Openbook Assi.- 10+ Article Review - 15+ Attendance - 5)
4. 80% Attendance



Basics of Networking & Socket Programming

Ms.Shweta Rajput

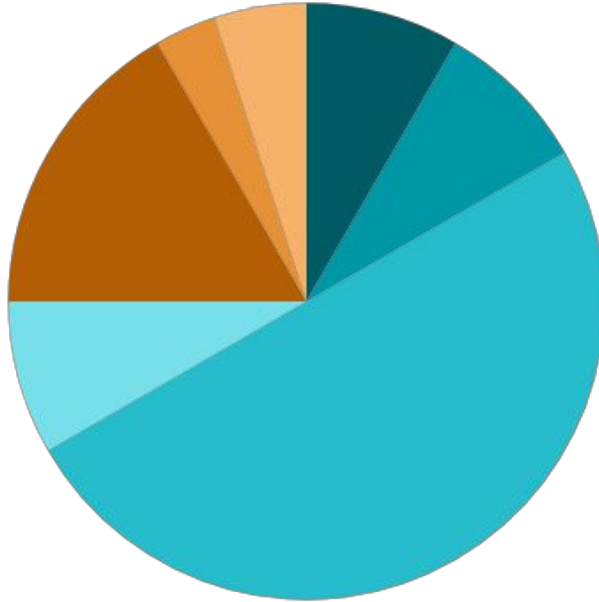
Department of Computer Science & Engineering



Outline

1. Network Basics and Socket overview
2. java.net package
3. InetAddress, Socket, ServerSocket
4. TCP/IP client sockets, TCP/IP server sockets
5. Datagram-DatagramPacket, DatagramSocket
6. URL, URL Connection

Lecture Structure



05 Minutes

Outline & Objectives



05 Minutes

Recap of Previous Lecture



30 Minutes

Delivery of Actual Content as per Lesson Plan



05 Minutes

Industry Relevance



10 Minutes

Interactive Activities



02 Minutes

Importance of Discipline



03 Minutes

Recording of Attendance

Reference: Office Order No. Provost/092023135, Date: 18th September 2023



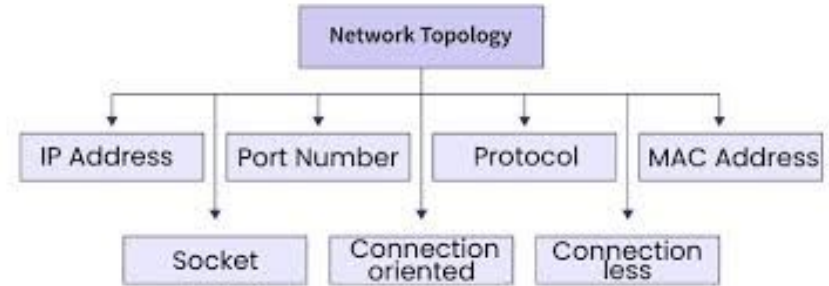
Objectives

1. Understand basic networking concepts and TCP/IP model
2. Learn the role of the java.net package in network programming
3. Work with InetAddress for IP address and host resolution
4. Access web resources using URL and URLConnection
5. Use Socket and ServerSocket for TCP client–server communication
6. Implement UDP communication using Datagram sockets
7. Differentiate between TCP and UDP communication models



Network Basics and Socket overview

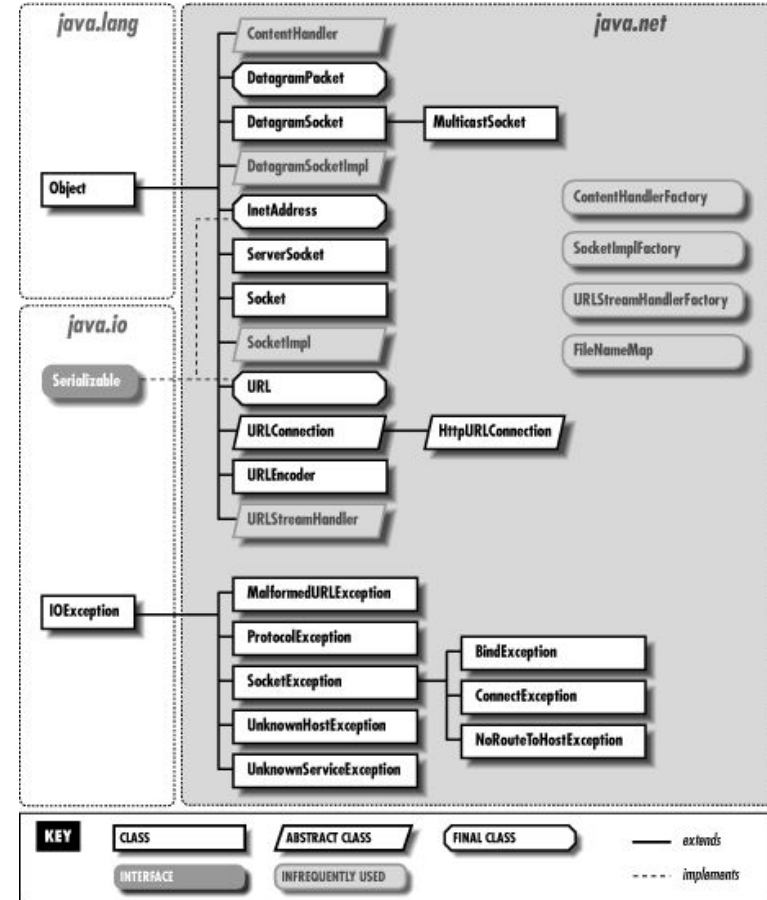
- IP Address - Uniquely identifies a machine on a network
- Port Number - Identifies a specific process/application on a machine
- Socket - Logical endpoint of communication between two machines
- Protocol - Set of rules used for data communication
- MAC Address - Physical hardware address of a network interface
- Connection-Oriented Communication : Communication established with a connection before data transfer (e.g., TCP)
- Connection-Less Communication : Communication without establishing a connection (e.g., UDP)





java.net Package

- Provides the classes for implementing networking applications.





InetAddress

- Represents an Internet Protocol (IP) address
- An IP address is either a 32-bit or 128-bit unsigned number used by IP, a lower-level protocol on which protocols like UDP and TCP are built
- An instance of an InetAddress consists of an IP address and possibly its corresponding host name
- InetAddress can handle both IPv4 and IPv6 addresses



InetAddress

Modifier and Type	Method	Description
<code>byte[]</code>	<code>getAddress()</code>	Returns the raw IP address of this <code>InetAddress</code> object.
<code>static InetAddress</code>	<code>getByName(String host)</code>	Determines the IP address of a host, given the host's name.
<code>String</code>	<code>getHostAddress()</code>	Returns the IP address string in textual presentation.
<code>String</code>	<code>getHostName()</code>	Gets the host name for this IP address.
<code>static InetAddress</code>	<code>getLocalHost()</code>	Returns the address of the local host.



Programming Time : Demo InetAddress

```
// Demonstrate InetAddress.
import java.net.*;

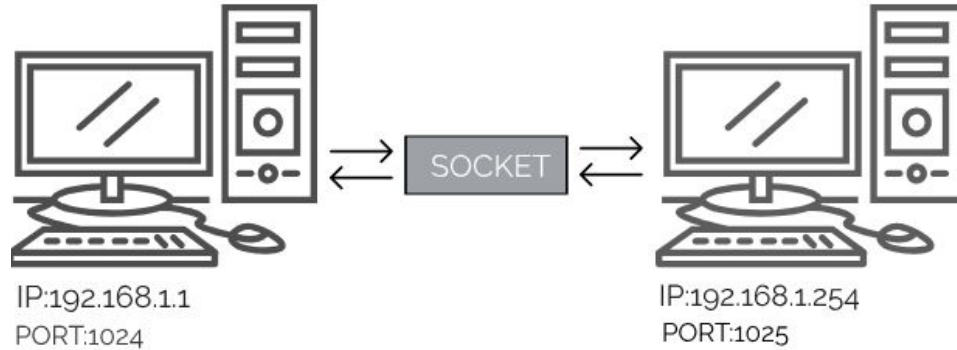
class InetAddressTest
{
    public static void main(String args[]) throws UnknownHostException {
        InetAddress Address = InetAddress.getLocalHost();
        System.out.println(Address);
        Address = InetAddress.getByName("osborne.com");
        System.out.println(Address);
        InetAddress SW[] = InetAddress.getAllByName("www.nba.com");
        for (int i=0; i<SW.length; i++)
            System.out.println(SW[i]);
    }
}
```

Here is the output produced by this program. (Of course, the output you see may be slightly different.)

```
default/206.148.209.138
osborne.com/198.45.24.162
www.nba.com/64.5.96.214
www.nba.com/64.5.96.216
```

Source of Image: Java complete Reference Page no 602

TCP/IP Sockets



- There are two kinds of TCP sockets in Java. One is for servers, and the other is for clients.
- The `ServerSocket` class is designed to be a “listener,” which waits for clients to connect before doing anything. Thus, `ServerSocket` is for servers.
- The `Socket` class is for clients. It is designed to connect to server sockets and initiate protocol exchanges.



Socket

- This class implements client sockets (also called just "sockets"). A socket is an endpoint for communication between two machines.

Constructor	Description
<code>Socket ()</code>	Creates an unconnected socket, with the system - default type of SocketImpl.
<code>Socket(InetAddress address, int port)</code>	Creates a stream socket and connects it to the specified port number at the specified IP address.
<code>Socket(String host, int port)</code>	Creates a stream socket and connects it to the specified port number on the named host.
<code>Socket(String host, int port, InetAddress localAddr, int localPort)</code>	Creates a socket and connects it to the specified remote host on the specified remote port.



Socket

Modifier and Type	Method	Description
void	<code>close ()</code>	Closes this socket.
InetAddress	<code>getInetAddress ()</code>	Returns the address to which the socket is connected.
InputStream	<code>getInputStream ()</code>	Returns an input stream for this socket.
OutputStream	<code>getOutputStream ()</code>	Returns an output stream for this socket.
boolean	<code>isConnected ()</code>	Returns the connection state of the socket.
int	<code>getPort ()</code>	Returns the remote port number to which this socket is connected.
int	<code>getLocalPort ()</code>	Returns the local port number to which this socket is bound.



ServerSocket

- This class implements server sockets. A server socket waits for requests to come in over the network.
- It performs some operation based on that request, and then possibly returns a result to the requester.

Constructor	Description
<code>ServerSocket()</code>	Creates an unbound server socket.
<code>ServerSocket(int port)</code>	Creates a server socket, bound to the specified port.



ServerSocket

- This class implements server sockets. A server socket waits for requests to come in over the network.
- It performs some operation based on that request, and then possibly returns a result to the requester.

Constructor	Description
<code>ServerSocket()</code>	Creates an unbound server socket.
<code>ServerSocket(int port)</code>	Creates a server socket, bound to the specified port.

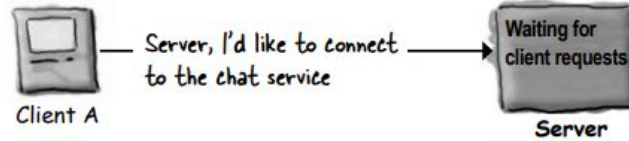


ServerSocket

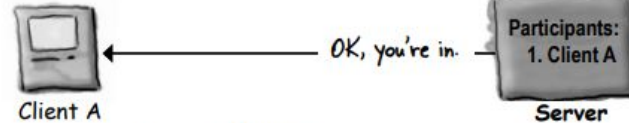
Modifier and Type	Method	Description
Socket	accept()	Listens for a connection to be made to this socket and accepts it.
void	bind(SocketAddress endpoint)	Binds the <code>ServerSocket</code> to a specific address (IP address and port number).
void	close()	Closes this socket.
InetAddress	getInetAddress()	Returns the local address of this server socket.
int	getLocalPort()	Returns the port number on which this socket is listening.
SocketAddress	getLocalSocketAddress()	Returns the address of the endpoint this socket is bound to

How it Works:

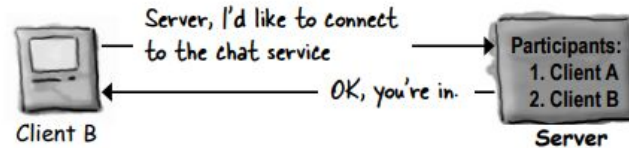
- 1 Client connects to the server



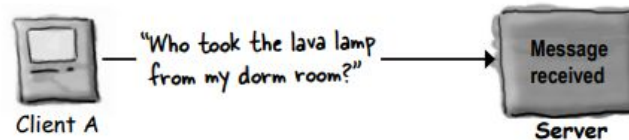
- 2 The server makes a connection and adds the client to the list of participants



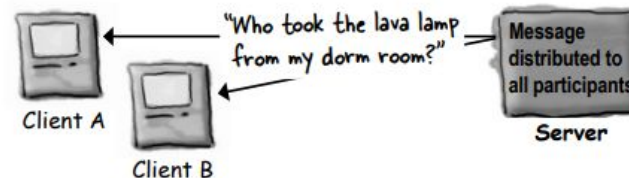
- 3 Another client connects



- 4 Client A sends a message to the chat service



- 5 The server distributes the message to ALL participants (including the original sender)

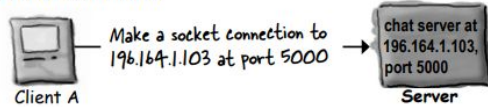


Source of Image: Head First Java Page no 473

Client-Server Programming Time : 1. Check Connection

1 Connect

Client connects to the server by establishing a **Socket** connection.



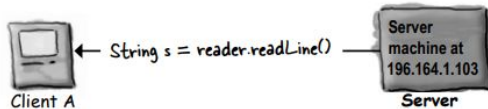
2 Send

Client **sends** a message to the server



3 Receive

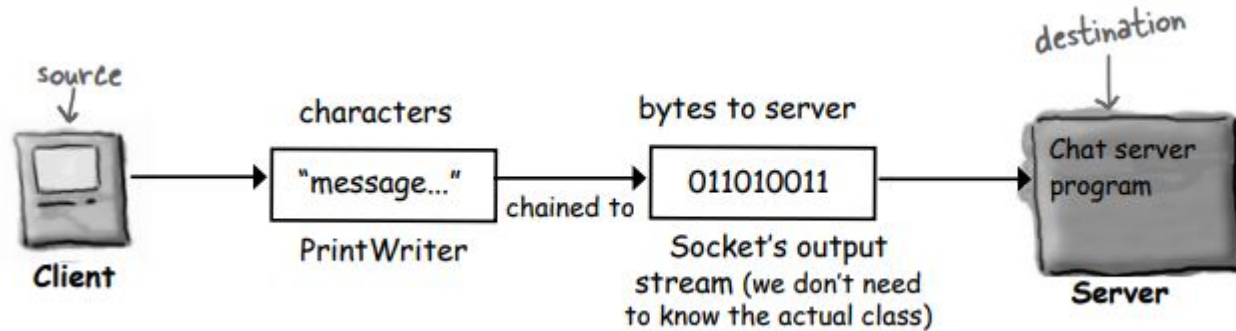
Client **gets** a message from the server



Source of Image: Head First Java Page no 474

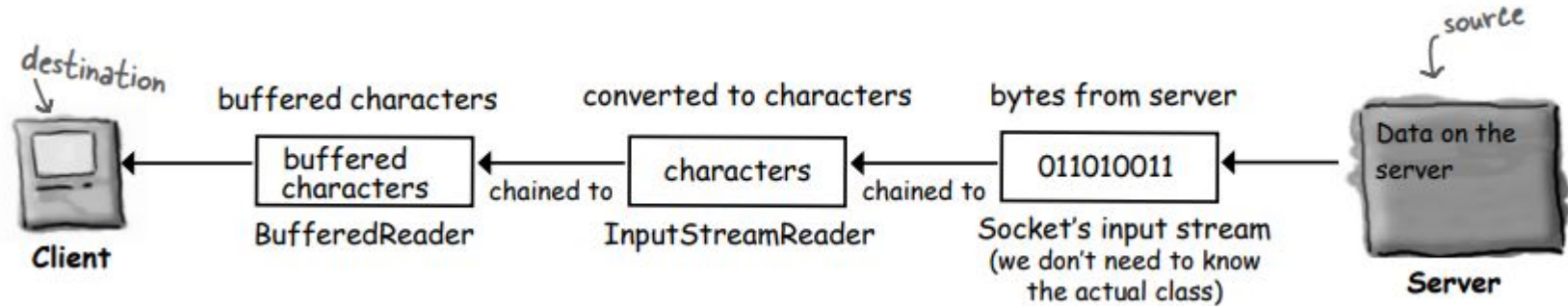


Client-Server Programming Time : 2. Client Send Message To Server



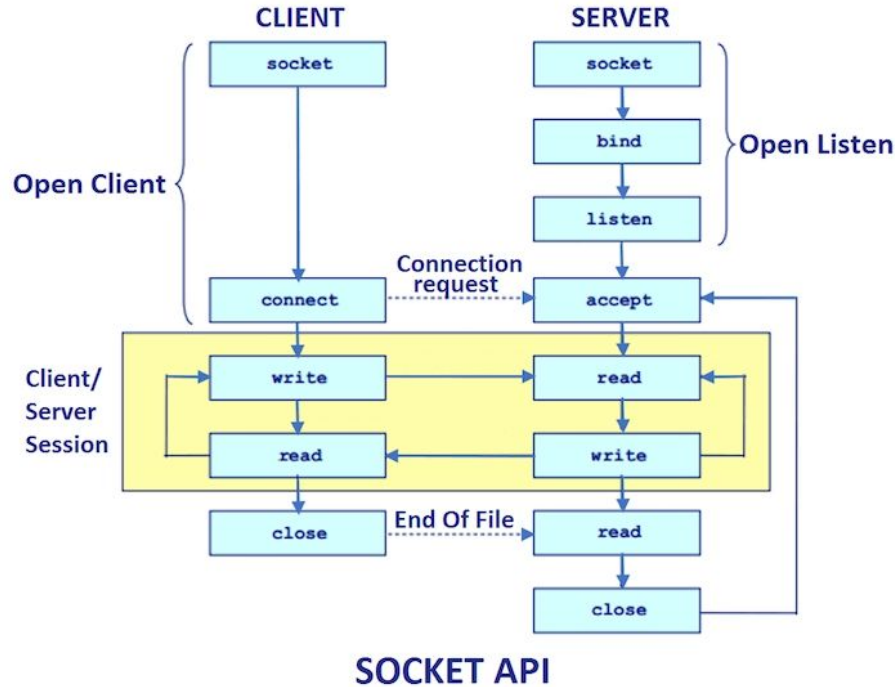
Source of Image: Head First Java Page no 479

Client-Server Programming Time : 3. Server Reply to Client



Source of Image: Head First Java Page no 478

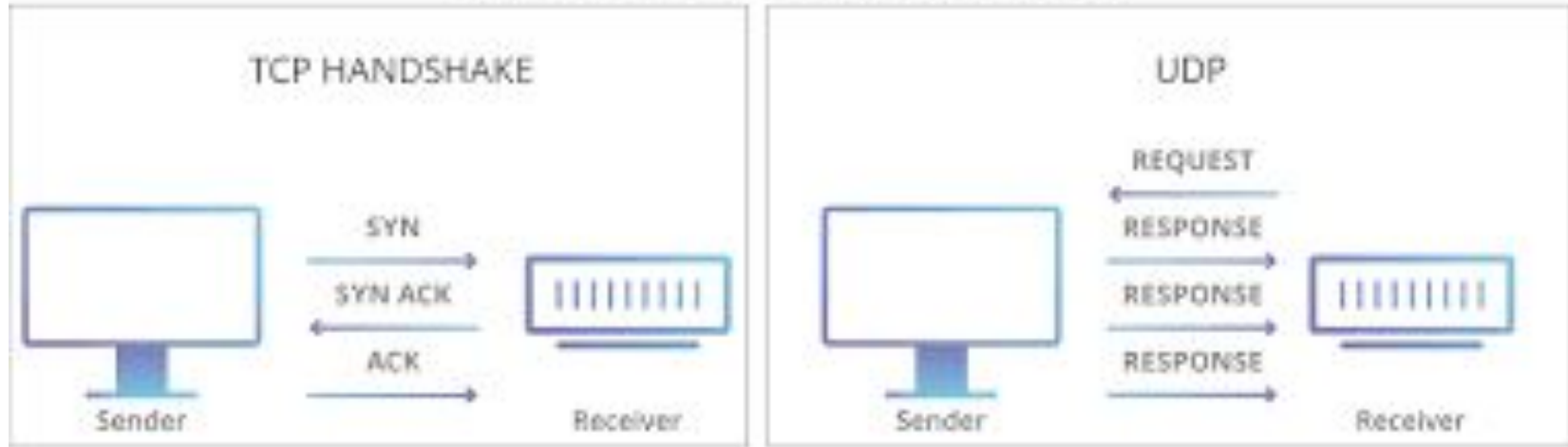
Client-Server Programming Time : 4. Chat Application



Source of Image: Head First Java Page no 473



TCP Vs UDP Protocol



Source of Image: Head First Java Page no 473



Datagrams

- Datagrams are bundles of information passed between machines.
- Once the datagram has been released to its intended target, there is no assurance that it will arrive or even that someone will be there to catch it.
- Likewise, when the datagram is received, there is no assurance that it hasn't been damaged in transit or that whoever sent it is still there to receive a response.
- Java implements datagrams on top of the UDP protocol by using two classes:
 - the DatagramPacket object is the data container,
 - while the DatagramSocket is the mechanism used to send or receive the DatagramPackets.

Source of Image: Head First Java Page no 473



DatagramSocket

- This class represents a socket for sending and receiving datagram packets.
- A datagram socket is the sending or receiving point for a packet delivery service.
- Each packet sent or received on a datagram socket is individually addressed and routed. Multiple packets sent from one machine to another may be routed differently, and may arrive in any order.

Constructor	Description
<code>DatagramSocket()</code>	Constructs a datagram socket and binds it to any available port on the local host machine.
<code>DatagramSocket(int port)</code>	Constructs a datagram socket and binds it to the specified port on the local host machine.
<code>DatagramSocket(int port, InetAddress laddr)</code>	Creates a datagram socket, bound to the specified local address.

DatagramSocket

Modifier and Type	Method	Description
void	<code>close ()</code>	Closes this datagram socket.
InetAddress	<code>getInetAddress ()</code>	Returns the address to which this socket is connected.
int	<code>getPort ()</code>	Returns the port number to which this socket is connected.
void	<code>receive (DatagramPacket p)</code>	Receives a datagram packet from this socket.
void	<code>send (DatagramPacket p)</code>	Sends a datagram packet from this socket



DatagramPacket

- Datagram packets are used to implement a connectionless packet delivery service.
- Each message is routed from one machine to another based solely on information contained within that packet.
- Multiple packets sent from one machine to another might be routed differently, and might arrive in any order. Packet delivery is not guaranteed.
- Two Kinds of DatagramPacket
 - Send
 - Receive

Source of Image: Head First Java Page no 473



DatagramPacket

- Datagram packets are used to implement a connectionless packet delivery service.
- Each message is routed from one machine to another based solely on information contained within that packet.
- Multiple packets sent from one machine to another might be routed differently, and might arrive in any order. Packet delivery is not guaranteed.
- Two Kinds of DatagramPacket
 - Sending data over a DatagramSocket
 - Receiving data over a DatagramSocket

Source of Image: Head First Java Page no 473



DatagramPacket

Constructor	Description
DatagramPacket (byte[] buf, int length)	Constructs a DatagramPacket for receiving packets of length length.
DatagramPacket (byte[] buf, int length, InetAddress address, int port)	Constructs a datagram packet for sending packets of length length to the specified port number on the specified host.
DatagramPacket (byte[] buf, int offset, int length)	Constructs a DatagramPacket for receiving packets of length length, specifying an offset into the buffer.
DatagramPacket (byte[] buf, int offset, int length, InetAddress address, int port)	Constructs a datagram packet for sending packets of length length with offset ioffset to the specified port number on the specified host.

Source of Image: Head First Java Page no 473

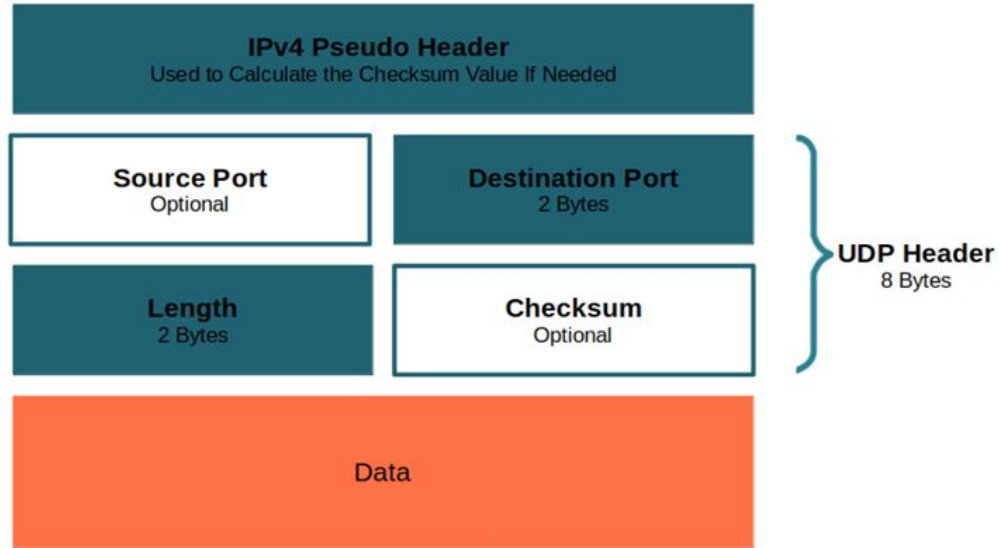


DatagramPacket

Modifier and Type	Method	Description
InetAddress	getAddress ()	Returns the IP address of the machine to which this datagram is being sent or from which the datagram was received.
byte[]	getData ()	Returns the data buffer.
int	getLength ()	Returns the length of the data to be sent or the length of the data received.
int	getPort ()	Returns the port number on the remote host to which this datagram is being sent or from which the datagram was received.
void	setAddress (InetAddress iaddr)	Sets the IP address of the machine to which this datagram is being sent.
void	setData (byte[] buf)	Set the data buffer for this packet.



DatagramPacket



Source of Image: Head First Java Page no 473



Client-Server Programming Time : 5. Send Message From Client To Server

Source of Image: Head First Java Page no 473



Client-Server Programming Time : 6. Chat Application

Source of Image: Head First Java Page no 473



URI

- Identifies a resource
- Can be a name, location, or both
- Used mainly for parsing and identification
- May not provide a way to access the resource
- A URI is a uniform resource identifier while a URL is a uniform resource locator.
- Every URL is a URI, abstractly speaking, but not every URI is a URL.
- A URI may be either absolute or relative, while a URL must be absolute

Source of Image: Head First Java Page no 473



URI

Constructor	Description
<code>URI (String str)</code> Constructs a URI by parsing the given string.	<code>URI (String str)</code> Constructs a URI by parsing the given string.

Modifier and Type	Method	Description
<code>URL</code>	<code>toURL ()</code>	Constructs a URL from this URI.



URL

- Class URL represents a Uniform Resource Locator, a pointer to a "resource" on the World Wide Web.
- A resource can be something as simple as a file or a directory, or it can be a reference to a more complicated object, such as a query to a database or to a search engine.
- `http://www.example.com:1080/docs/resource1.html`
- A URL specification is based on four components.
 - The first is the protocol to use. Common protocols are HTTP, FTP, gopher, file
 - The second component is the host name or IP address of the host to use
 - The third component, the port number, is an optional parameter
 - The fourth part is the actual file path

Source of Image: Head First Java Page no 473



URL

Modifier and Type	Method	Description
Object	getContent ()	Gets the contents of this URL.
int	getDefaultPort ()	Gets the default port number of the protocol associated with this URL.
String	getFile ()	Gets the file name of this URL.
String	getHost ()	Gets the host name of this URL, if applicable.
String	getPath ()	Gets the path part of this URL.
int	getPort ()	Gets the port number of this URL.
String	getProtocol ()	Gets the protocol name of this URL.
URLConnection	openConnection ()	Returns a URLConnection instance that represents a connection to the remote object referred to by the URL.



Programming Time : Demo URL



URLConnection

- URLConnection is a general-purpose class for accessing the attributes of a remote resource.
- Once you make a connection to a remote server, you can use URLConnection to inspect the properties of the remote object before actually transporting it locally.

Constructor	Description
<code>URLConnection(URL url)</code>	Constructs a URL connection to the specified URL.



URLConnection

Modifier and Type	Method	Description
int	<code>getLength()</code>	Returns the value of the <code>content-length</code> header field.
long	<code>getLengthLong()</code>	Returns the value of the <code>content-length</code> header field as a long.
String	<code>getContentType()</code>	Returns the value of the <code>content-type</code> header field.
long	<code>getDate()</code>	Returns the value of the <code>date</code> header field.
long	<code>getExpiration()</code>	Returns the value of the <code>expires</code> header field.
InputStream	<code>getInputStream()</code>	Returns an input stream that reads from this open connection.
long	<code>getLastModified()</code>	Returns the value of the <code>last-modified</code> header field.
OutputStream	<code>getOutputStream()</code>	Returns an output stream that writes to this connection.



Programming Time : Demo URLConnection

 **Thank You..**